Finally, the last template in our simple blog will be the home page. It's in a different location because it's the home page and so it should be at *app/index.html*:

```
---
layout: default
description: Your Beautiful Blog
---

<h1> Recent Posts </h1>
{% for post in site.posts limit:50 %}
<h2> <a href="{{ post.url }}">{{ post.title }}</a> </h2>
<div class="preview">
    {% if post.description %}
    <span class="body">{{ post.description | strip_html |
truncatewords: 300 }}</span>
    {% else %}
    <span class="body">{{ post.excerpt | strip_html }}</span>
    {% endif %}
</div>
{% endfor %}
```

On the home page, we'll just list the 50 most recent articles with an excerpt from each one. You may have noticed that we are again extending from the default layout.

We're nearly ready to generate the site but need one more thing...

Using Gems plugins

Remember we mentioned plugins? We'll configure them now. Create a Gemfile and put this in it, as shown below:

```
source 'https://rubygems.org'

group :jekyll_plugins do
    gem 'jekyll', '~>3.0'
    gem 'kramdown'
    gem 'rdiscount'
    gem 'jekyll-sitemap'
    gem 'jekyll-redirect-from'
end
```

We'll install these Gems later when we generate the site. At this point, you might want to write a few words of your first blog post and put it into *app/_posts/*. Let's generate our site, now!

We're going to use a Docker container with Jekyll already installed on it. You can of course install Jekyll locally, but using a Docker container makes it more portable; for example, you might decide to build your blog continuously later on, by containing your tools inside Docker containers. You don't have to install them on every server you want to build the blog on.

Run this to install the Gems and build your shiny new blog, as follows:

```
$ docker run --rm \
        -v "$(pwd):/src" \
        -w /src \
        ruby:2.3 \
        sh -c 'bundle install \
            --path vendor/bundle \
            && exec jekyll build --watch'
```

We've added the *–watch* flag at the end, which is handy when we're making changes and we want to see them reflected immediately on the blog.

Voila! Have a look in the Web/folder—you should see lots of HTML files, which are what Jekyll generated. If you were to FTP that entire Web/folder to a server, you would have a working blog, but we're going to put it into a Docker container.

The Dockerfile

Our container will be super simple. We just need to serve the Web/folder that we just generated. Here's our Dockerfile:

```
FROM nginx
EXPOSE 80
COPY web/ /usr/share/nginx/html
```

That's it! Now, let's build and run the image:

```
$ docker build -t my-shiny-blog .
$ docker run -d \
        -p 80:80 \
        -v "$(pwd)/web:/usr/share/nginx/html" \
        my-shiny-blog
```

Now hit 127.0.0.1:8080 in your browser to see your blog in action!

Notice that we've mounted the source into the container as a volume, which will allow us to see updates in real-time when Jekyll regenerates the site (since Jekyll is still running with '–watch'), without having to rebuild the image again. Before pushing the image to your registry, just remember to rebuild!

That's it! We can of course get fancy and minimise Javascript or compile less to css using Gulp, but we'll leave that as an exercise for the reader. The trick is to put less source code outside of the app/directory and have Gulp place the final versions in the app/directory. That way, Jekyll will copy them over when the site is generated. END

References

[1] *https://jekyllrb.com/*
[2] *https://github.com/jekyll/docker*

By: Srijan Agarwal

The author is a developer at *WikiToLearn*, an open source enthusiast and works with JS and PHP mostly. You can contact him at *www.srijanagarwal.me*.